

Klient HTTP w Go

Termin oddania: do końca semestru

Maksymalna ocena: 10 punktów

Cel zadania

Celem zadania jest zbudowanie małego klienta HTTP w języku Go, który komunikuje się z fikcyjną usługą JSON API.

Klient powinien obsługiwać dwa endpointy fikcyjnej usługi:

`GET /items`

oraz

`POST /items`

Zadanie ma na celu przećwiczenie poprawnego użycia pakietów `net/http`, `context`, `encoding/json` oraz `net/http/httptest`.

Opis zadania

Należy zaimplementować klienta API, który:

- korzysta z wielokrotnie używanego `http.Client`,
- tworzy żądania świadome kontekstu,
- koduje dane do JSON,
- dekoduje odpowiedzi JSON,
- jawnie obsługuje statusy HTTP,
- korzysta z własnego transportu dodającego nagłówek `User-Agent`,
- posiada testy napisane z użyciem `httptest.Server`.

Fikcyjna usługa API powinna zostać zasymulowana w testach za pomocą `httptest.Server`.

Endpointy fikcyjnej usługi

`GET /items`

Endpoint służy do pobrania listy elementów.

Przykładowa odpowiedź API:

```
[
  {
    "id": 1,
    "name": "First item",
    "description": "Example description"
  },
  {
    "id": 2,
    "name": "Second item",
    "description": "Another example description"
  }
]
```

Klient powinien udostępnić metodę podobną do:

```
func (c *Client) GetItems(ctx context.Context) ([]Item, error)
```

Metoda powinna:

- wysłać żądanie GET /items,
- użyć przekazanego context.Context,
- sprawdzić, czy status odpowiedzi to 200 OK,
- zdekodować odpowiedź JSON do struktury Go,
- zwrócić błąd w przypadku niepoprawnego statusu lub błędnego JSON-a.

POST /items

Endpoint służy do utworzenia nowego elementu.

Przykładowe ciało żądania:

```
{
  "name": "New item",
  "description": "Description of the new item"
}
```

Przykładowa odpowiedź API:

```
{
  "id": 3,
  "name": "New item",
  "description": "Description of the new item"
}
```

Klient powinien udostępniać metodę podobną do:

```
func (c *Client) CreateItem(ctx context.Context, input CreateItemReq
```

Metoda powinna:

- zakodować dane wejściowe do JSON,
- wysłać żądanie POST /items,
- ustawić nagłówek Content-Type: application/json,
- użyć przekazanego context.Context,
- sprawdzić, czy status odpowiedzi to 201 Created,
- zdekodować odpowiedź JSON do struktury Go,
- zwrócić błąd w przypadku niepoprawnego statusu, błędnego JSON-a lub błędu żądania.

Przykładowe struktury danych

```
type Item struct {
    ID          int    `json:"id"`
    Name        string `json:"name"`
    Description string `json:"description"`
}

type CreateItemRequest struct {
    Name        string `json:"name"`
    Description string `json:"description"`
}
```

Wymagania techniczne

1. Wielokrotnie używany http.Client

Klient API powinien korzystać z jednego obiektu http.Client, który jest używany wielokrotnie.

Nie należy tworzyć nowego `http.Client` przy każdym żądaniu.

Przykładowa struktura:

```
type Client struct {  
    baseURL    string  
    httpClient *http.Client  
}
```

2. Żądania świadome kontekstu

Wszystkie żądania HTTP powinny być tworzone za pomocą:

```
http.NewRequestWithContext(ctx, method, url, body)
```

Dzięki temu możliwe jest anulowanie żądania lub ustawienie limitu czasu z poziomu kontekstu.

Przykład użycia:

```
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)  
defer cancel()  
  
items, err := client.GetItems(ctx)
```

3. Kodowanie i dekodowanie JSON

Klient powinien korzystać z pakietu `encoding/json`.

Przy żądaniu POST należy zakodować strukturę Go do JSON.

Przy żądaniach GET i POST należy zdekodować odpowiedź JSON do odpowiednich struktur Go.

4. Jawna obsługa statusów HTTP

Klient powinien jawnie sprawdzać statusy HTTP.

Oczekiwane statusy:

Metoda	Endpoint	Oczekiwany status
--------	----------	-------------------

GET	/items	200 OK
POST	/items	201 Created

W przypadku innego statusu klient powinien zwrócić czytelny błąd.

Przykład:

```
if resp.StatusCode != http.StatusOK {  
    return nil, fmt.Errorf("unexpected status code: %d", resp.Status)  
}
```

5. Własny transport dodający User-Agent

Należy zaimplementować własny transport HTTP, który dodaje nagłówek User-Agent do każdego żądania.

Przykładowa wartość nagłówka:

```
User-Agent: go-http-clientPV
```

Transport powinien opakowywać istniejący `http.RoundTripper`.

Przykład struktury:

```
type UserAgentTransport struct {  
    Base      http.RoundTripper  
    UserAgent string  
}
```

Transport powinien implementować metodę:

```
func (t *UserAgentTransport) RoundTrip(req *http.Request) (*http.Re:
```

Jeśli Base nie zostanie ustawiony, transport może używać `http.DefaultTransport`.

Testy

Projekt powinien zawierać testy z użyciem `httptest.Server`.

Testy powinny sprawdzać co najmniej:

1. Czy metoda GetItems wysyła żądanie GET /items.
2. Czy metoda GetItems poprawnie dekoduje odpowiedź JSON.
3. Czy metoda CreateItem wysyła żądanie POST /items.
4. Czy metoda CreateItem poprawnie koduje ciało żądania JSON.
5. Czy klient poprawnie obsługuje nieoczekiwane statusy HTTP.
6. Czy do żądań dodawany jest nagłówek User-Agent.
7. Czy żądania korzystają z przekazanego context.Context.

Przykładowy schemat testu:

```
func TestGetItems(t *testing.T) {
    server := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        if r.Method != http.MethodGet {
            t.Errorf("expected GET, got %s", r.Method)
        }

        if r.URL.Path != "/items" {
            t.Errorf("expected /items, got %s", r.URL.Path)
        }

        w.Header().Set("Content-Type", "application/json")
        w.WriteHeader(http.StatusOK)
        _, _ = w.Write([]byte(`{"id":1,"name":"First item","description":"First item"}`))
    }))
    defer server.Close()

    client := NewClient(server.URL)

    items, err := client.GetItems(context.Background())
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }

    if len(items) != 1 {
        t.Fatalf("expected 1 item, got %d", len(items))
    }
}
```

Kryteria oceny

Maksymalna liczba punktów: 10

Element	Punkty
Poprawna struktura klienta API	1

Wielokrotnie używany <code>http.Client</code>	1
Obsługa endpointu GET <code>/items</code>	1
Obsługa endpointu POST <code>/items</code>	1
Kodowanie i dekodowanie JSON	2
Użycie <code>context.Context</code> w żądaniach	1
Jawna obsługa statusów HTTP	1
Własny transport dodający User-Agent	1
Testy z użyciem <code>httptest.Server</code>	1

Materiały pomocnicze

Oficjalna dokumentacja

- [net/http/httptest — Go Packages](#)
Oficjalna dokumentacja pakietu `httptest`, w tym `httptest.NewServer`.
- [net/http — Go Packages](#)
Oficjalna dokumentacja pakietu `net/http`, w tym `http.Client`, `http.NewRequestWithContext`, `RoundTripper` i `Transport`.
- [encoding/json — Go Packages](#)
Oficjalna dokumentacja pakietu do kodowania i dekodowania JSON.
- [context — Go Packages](#)
Oficjalna dokumentacja pakietu `context`.

Tutoriale i artykuły

- [Unit Testing HTTP Client in Go](#)
Praktyczny przykład testowania klienta HTTP z użyciem `httptest.Server`.
 - [Creating a Test HTTP Server in Go](#)
Krótkie wprowadzenie do `httptest.NewServer`.
 - [Using httptest.Server in Go to Mock and Test External API Calls](#)
Artykuł o mockowaniu zewnętrznych API za pomocą `httptest.Server`.
 - [Testing Golang with httptest Best Practices](#)
Szerszy opis praktyk testowania HTTP w Go.
 - [Golang httptest Example](#)
Przykłady użycia `httptest`, w tym testowanie klientów HTTP.
-